SOLUTION DOCUMENTATION

Learning Pathfinder

An AI-powered personalised learning path recommender. It turns a sentence about where you want to end up into a sequenced roadmap of real courses — one that explains itself, tracks what you finish, and rebuilds when you say it got something wrong.

Stack React 18 · FastAPI · SQLite · Groq **Model** openai/gpt-oss-120b **Retrieval** Okapi BM25, pure Python

Live app learning-pathfinder.streamlit.app **Source** github.com/Yaswanth-9505/learning-pathfinder

01 Problem understanding

Course discovery is a solved problem. Course *sequencing* is not. A learner searching "backend engineering" gets thousands of results ranked by popularity, each one a reasonable answer to "what could I watch next" and none of them an answer to "what should I do first, and what after that."

The gap has four distinct parts, and each needs a different mechanism:

THE GAP	WHY RANKING ALONE FAILS	WHAT IT NEEDS
Ordering	Relevance is symmetric; prerequisites are not. A ranker has no notion of "before".	A dependency-aware sequence
Starting point	Two learners with the same goal need different first steps.	A model of what they already know
Justification	"Because people like you watched it" is not a reason a learner can act on.	A stated rationale per item
Staleness	A plan fixed at signup is wrong by week three.	Re-planning from progress and feedback

A one-size-fits-all path fails because the same goal decomposes differently depending on the starting point. "Become a backend engineer" is nine courses for someone who has never opened a terminal and four for

someone who already ships Python. The unit of personalisation is not the course — it is the **gap between the learner and the goal.**

02 Solution approach

The obvious build is to ask an LLM for a learning path and render what comes back. We tried that first, and it fails in a specific, disqualifying way: the model invents courses. It produces plausible titles attached to providers that never published them, and URLs that 404. For a recommender whose entire output is "go do this thing," a fabricated resource is not a cosmetic bug.

So the system inverts the usual arrangement. The catalog is the source of truth and the LLM is a curator working over it. It never supplies a course fact — only selection, ordering and rationale. Anything it names that is not in the catalog is dropped before it reaches the UI.

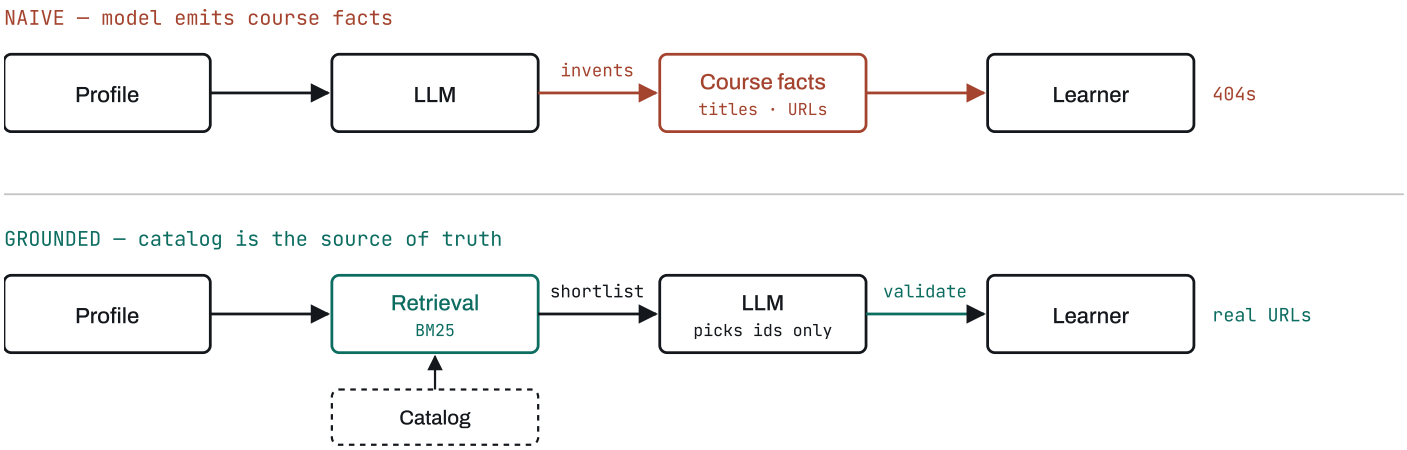

FIGURE 1 · THE ONE EDGE THAT CHANGES

In the naive design the arrow carrying course facts originates at the model, so nothing downstream can tell an invented course from a real one. In the grounded design that arrow originates at the catalog; the model's output is reduced to a list of ids, which is checkable. Everything else is identical.

This buys three properties that matter more than the extra moving parts cost: every displayed resource is real, every recommendation can be traced to a specific missing skill, and the catalog can be corrected or extended without touching a prompt.

03 System architecture

Three tiers. A React client, a FastAPI service holding all the logic, and two stores — a read-only JSON corpus and a SQLite database for everything that changes. The only external dependency is the Groq inference API.

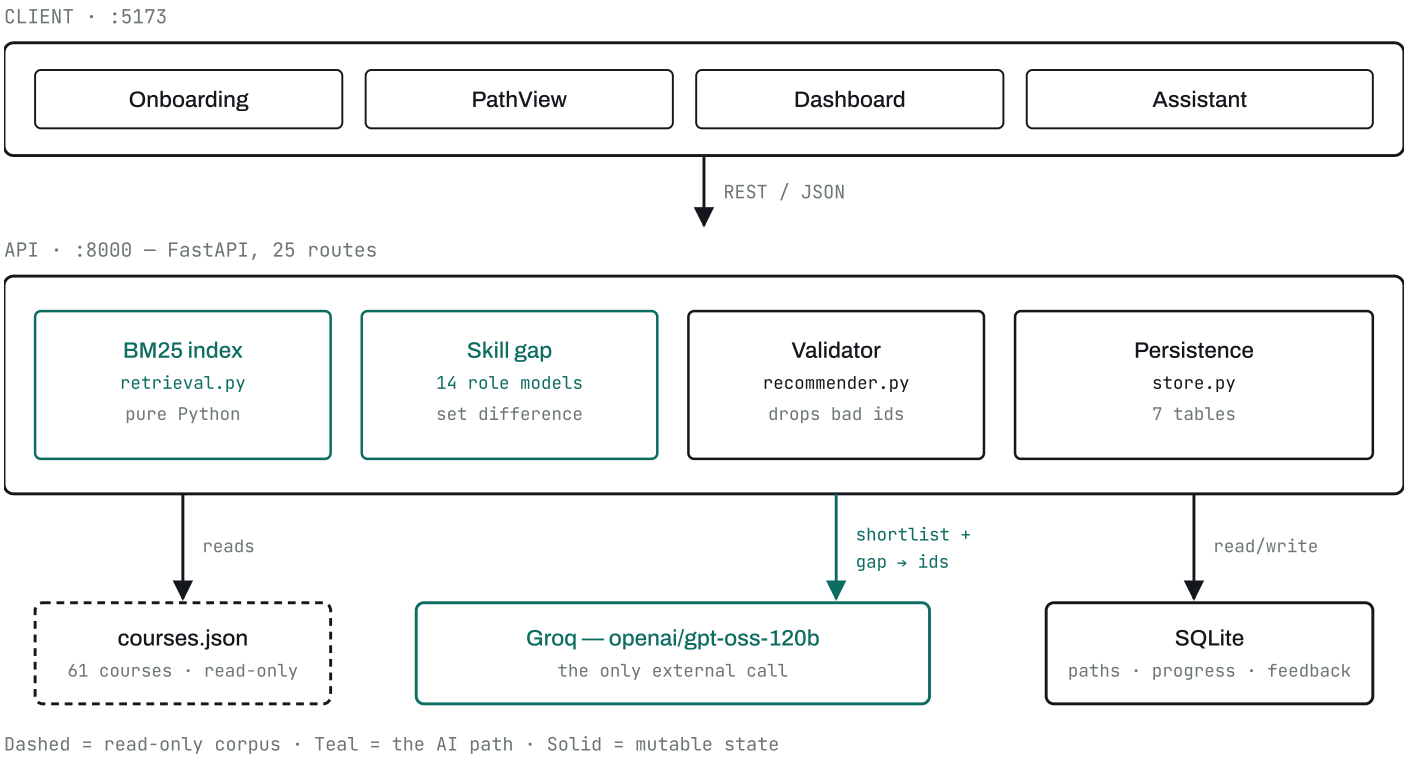

FIGURE 2 · COMPONENT AND DATA FLOW

The model sits beside the system rather than in front of its data: it receives a shortlist and a skill gap and returns ids, and every write to durable state passes through the validator first.

Two surfaces, one core

The application ships two front ends over the same logic. A React and FastAPI pair gives the full-fidelity UI plus a REST API; a Streamlit app gives a single-process build that deploys to Streamlit Community Cloud in one click. Both call `engine/service.py` directly, so neither owns business rules and the two cannot drift into different behaviour. The engine itself is pure standard library — no framework imports at all — which is what made the second surface cheap to add rather than a rewrite.

Data model

Seven tables. Paths are versioned rather than mutated — adapting a path writes a new version and deactivates the old one, so the history of how someone's plan evolved stays intact and is auditable.

TABLE	HOLDS	NOTABLE
`learners`	Profile, level, goals, weekly hours	—
`paths`	Generated roadmap, milestones, frozen skill gap	Versioned; `adapted_from` self-reference
`path_courses`	Ordered courses, reason, project, completion	Reason stored per course, not regenerated
`learner_skills`	Acquired skills	Auto-credited on completion; unique per learner
`feedback`	Signals and comments	Feeds the next generation
`assessments`	Milestone checkpoints and their question sets	One per milestone; answer key never leaves the server
`assessment_attempts`	Scored attempts	Best attempt decides skill credit
`chat_messages`	Assistant history	Last 6 turns re-injected as context

04 AI/ML techniques

Six stages run on every generation. Three are classical information retrieval and set logic; two are LLM calls; one is deterministic validation. The split is deliberate — anything that can be computed reliably is computed, and the model is reserved for the judgement calls that genuinely need it.

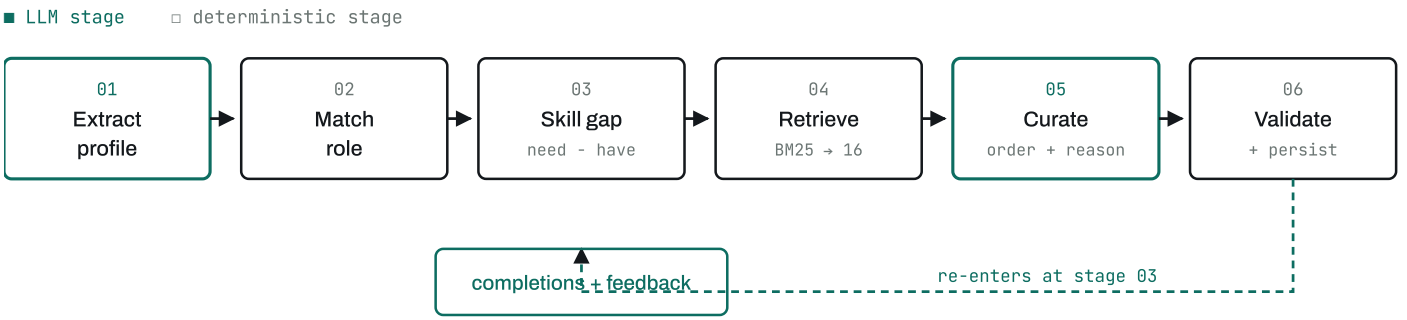

FIGURE 3 · GENERATION PIPELINE AND THE ADAPTATION LOOP

Progress and feedback re-enter at the skill-gap stage, which is why finishing a course visibly moves the gap and why a "too hard" signal changes what the next version contains.

Retrieval — Okapi BM25

Candidate selection is BM25 ($k_1 = 1.5$, $b = 0.75$) over a field-weighted document built per course: title and skills at weight 3, domains at 2, description and provider at 1. Query terms come from the goal (repeated, as the strongest signal), interests, and the missing skills.

Experience level biases rather than filters — a course one level away is scaled by 0.85, two levels by 0.72 — so a beginner can still be handed one stretch course when it is genuinely the right next step, instead of being walled into a beginner-only ghetto.

> WHY NOT EMBEDDINGS
>
> A sentence-transformer would add a multi-hundred-megabyte download and a warm-up cost to rank a 61-document corpus, and Groq serves no embedding model, so it would mean a second provider. At this corpus size lexical ranking is competitive, starts instantly, and — unlike a vector score — the ranking is inspectable term by term when a result looks wrong.

Skill-gap analysis

Fourteen target roles each carry a required-skill set. The learner's goal is matched to a role by alias-token overlap; their known skills are extracted from free text by whole-token matching against the catalog's 238-skill

vocabulary and unioned with skills auto-credited from completed courses. The gap is the set difference, and coverage is its complement — the number the dashboard leads with.

LLM usage

CALL	JOB	BUDGET
Profile extraction	Free text → typed profile, JSON mode	1,200
Path curation	Select, order, justify from the shortlist	3,000
Assistant	Answer grounded in the stored path	1,200
Checkpoint	5 application-level MCQs for a finished stage	2,600

The curation call returns ids and rationale only, never course metadata. That keeps the response small enough to fit the free tier's token ceiling and makes every field checkable against the catalog.

Does it actually recommend well?

A pipeline description is not evidence. Retrieval is therefore measured against a labelled set: 12 goals with 87 human relevance judgements (`eval/goldens.json`), scored offline with no API key so the numbers are reproducible by anyone who clones the repo.

METHOD	R@5	R@10	R@16	NDCG@10	MRR	SKILLCOV@16
BM25 (ours)	0.626	0.831	0.921	0.869	1.000	0.985
title-match	0.220	0.301	0.375	0.355	0.696	0.516
catalog order	0.041	0.146	0.221	0.126	0.217	0.321
random (5 seeds)	0.083	0.174	0.270	0.155	0.282	0.469

Recall@16 is the metric that matters. Sixteen is the shortlist the LLM receives, so it is a hard ceiling on what any generated path can contain - a course missed here can never appear in a roadmap no matter how good the prompt is. At 0.921 it runs 3.4x the random floor and 2.5x a title-matching baseline.

MRR of 1.000 means the top-ranked course is relevant for every labelled goal. **SkillCov@16 of 0.985** is the product metric rather than the IR one: the

shortlist teaches 98.5% of the skills the matched role requires, so the gap the path exists to close is almost always closable from the candidates on offer.

> GUARDED, NOT JUST MEASURED
>
> These thresholds are asserted in `tests/test_evaluation.py` and run in CI. A change to tokenizing, field weights or the BM25 constants that quietly degrades ranking fails the build instead of surfacing during a demo. One test checks the worst individual goal, not just the mean, because an average hides a single catastrophic query.

Deterministic post-processing

Prerequisites are *derived*, not requested: course B depends on earlier course A when A teaches a skill B builds on. Asking the model for prerequisites risks a graph that contradicts its own ordering; computing them from the ordering makes that class of inconsistency impossible. Validation also drops unknown ids, de-duplicates, clamps milestone indices into range, and backfills from retrieval rank if the model returns fewer than six courses.

---

05 Key features and workflows

61	14	238	28	60	0.92
COURSES	TARGET ROLES	SKILLS	API ROUTES	TESTS	RECALL@16

The primary workflow

1. **Describe the goal in natural language.** The learner types a sentence; the system extracts name, goal, interests, level, prior knowledge and available hours, then shows the parse back for correction. Getting it wrong is cheap — the profile is editable before anything is generated.
2. **See the gap before the plan.** The matched role and its missing skills are shown at review time, so the learner understands what the path is going to be built to close.
3. **Generate.** Retrieval, gap analysis and curation produce a roadmap grouped into milestones. Each course carries a rationale written for this

learner, its derived prerequisites, and a concrete project.

4. **Work and complete.** Ticking a course credits its skills automatically; coverage, milestone bars, hours and the estimated finish date all recompute from that single fact.
5. **Prove it.** Finishing every course in a stage unlocks that stage's checkpoint: five application-level multiple-choice questions generated from the courses actually taken. Grading happens server-side and the answer key is never sent to the browser. Passing at 70% credits the stage's skills with an `assessment:` source, so the dashboard can distinguish demonstrated skills from self-reported ones.
6. **Disagree.** Rate any course too hard, too easy, not relevant or helpful.
7. **Adapt.** Rebuilding re-runs the pipeline with the enlarged skill set and the feedback in the prompt, writing a new version.

> OBSERVED ADAPTATION
>
> On a backend-engineer profile with two courses completed, a `too_hard` signal on System Design Primer and a `not_relevant` on the DevOps course produced a v2 that dropped both flagged courses *and* both completed ones, and added FastAPI and API Design in their place — a shift from an infrastructure-weighted path to an API-weighted one.

Explainability

Every recommendation answers "why me, why now." The rationale is written at generation time and stored, so it never drifts. A dedicated endpoint returns the structured version: the skills the course closes, its position in the sequence, what it builds on, and the retrieval basis for the match. The assistant reads the same stored rationale, so its explanation and the card's explanation cannot disagree.

Dashboard

Coverage leads as a hero figure, with courses, hours, weeks remaining and skill coverage as a KPI row beneath. Milestone bars are direct-labelled with counts and paired with icons, so state never rests on colour alone; a table view is available for the same data. Chart colours were validated against the application's own dark surface rather than assumed.

06 Challenges faced

The model invented courses

The first working version asked the LLM to produce the whole path. It returned confident, well-structured JSON containing courses that do not exist. This is the failure that drove the entire architecture: the catalog became the source of truth and the model's output was reduced to ids that can be checked. The validator that drops unknown ids is covered by a test that feeds it a deliberately fabricated id.

A hard 8,000 token-per-minute ceiling

The free tier caps at 8,000 tokens/minute across all models. An early attempt at a 8,000-token completion budget failed with HTTP 413 before generating anything. Two changes brought it inside the envelope: the curation call returns ids rather than full course records, cutting output by roughly two-thirds, and candidates are serialised compactly at sixteen courses. Generation now runs at about 4,000 tokens end to end. The ceiling remains a real product constraint, not just a tuning detail — it caps the system at roughly one generation per minute for one user.

A silent substring bug in skill extraction

Skill matching originally used a naive substring test, which read the skill `c` out of the word "basic" — so a learner writing "basic Python" was credited with C. It was invisible in the UI and only surfaced when the extracted skill list was printed during testing. The fix was whole-token matching with padded boundaries for multi-word skills; the regression is now pinned by a named test.

The assessment generator was trivially gameable

The first working checkpoint looked excellent — application-level questions with plausible distractors, grounded in the right courses. Then the stored answer key read `[0, 0, 0, 0, 0]`. The model has a strong position bias and had put the correct option first in every single question, so answering "A" to everything scored 100% without reading a word. Nothing in the UI would have revealed this. Options are now shuffled server-side after

generation, seeded from the question text so a given question always lays out the same way, with `answer_index` remapped to follow the correct option. A test asserts that a fixed all-A submission fails.

Truncation looked like malformed JSON

Hitting the token limit mid-generation produced an unterminated string, which surfaced as a JSON parse error and sent debugging in the wrong direction. The pipeline now checks `finish_reason == "length"` explicitly and raises a message naming the real cause.

Adding Streamlit broke FastAPI

Installing Streamlit pulled `starlette` to 1.6.0, which the pinned `fastapi==0.104.1` cannot work with — every route registration died on `Router.__init__() got an unexpected keyword argument 'on_startup'`. The same dependency-rot failure mode as the Groq SDK below, surfacing a second time from a different direction. Resolved by moving the whole web stack to current releases (FastAPI 0.141, Pydantic 2.13) rather than pinning Streamlit backwards.

Dependency rot in a two-year-old scaffold

The starting scaffold pinned `groq==0.4.2`, which crashes on import against modern `httpx` because the `proxies` argument was removed. The originally configured model, `mixtral-8x7b-32768`, had been decommissioned by the provider and returned HTTP 400 on every call. The frontend used Create-React-App's `process.env` under Vite, which throws `ReferenceError` in the browser and white-screened the app, and a missing `postcss.config.js` meant no Tailwind class ever compiled. Each was independently fatal and none produced a message pointing at its own cause.

---

07 Scope and limits

Measured against the brief, this is what exists and what does not:

BUILT	Conversational natural-language intake with an editable parse

BUILT	Learner profiling: interests, level, prior learning, goals, capacity
BUILT	Recommendation engine over courses, projects and resources
BUILT	Path generator with derived prerequisites and milestones
BUILT	Assistant that explains recommendations and answers questions
BUILT	Dashboard: progress, skill development, milestones, next actions
BUILT	Feedback capture and path adaptation with version history
BUILT	Milestone checkpoint assessments with server-side grading and evidence-based skill credit
BUILT	Optional shared access gate for the deployed app, plus an opt-in learner roster
NOT BUILT	Per-user authentication — the gate admits everyone equally; learners are not isolated from one another
BUILT	Streamlit deployment path — single-process entry point, theme, secrets and runtime pin

HONEST CONSTRAINTS

The catalog is **curated, not crawled** — 61 courses across 26 domains, chosen so every skill each role requires is taught by something. A goal far outside those domains degrades to weak retrieval rather than a confident wrong answer, but it degrades. There is **no authentication**, so anyone holding a learner id can read that learner's data; this must not be deployed publicly as-is. The start screen's learner roster is therefore off by default (`PATHFINDER_SHOW_LEARNERS`), since listing it on an unauthenticated public URL would expose every learner's name and goal. On Streamlit Community Cloud the container filesystem is **ephemeral**, so the SQLite file is wiped on reboot, redeploy or sleep, taking every learner and path with it — `engine/store.py` is the only file that would change to move to hosted Postgres, but until that happens a deployed instance is a demo, not a product. And the free-tier token ceiling makes concurrent use impossible without a paid plan. Checkpoints are **multiple-choice only** — they test whether a learner recognises the right approach, not whether they can build the

> thing. The per-course projects address that, but nothing verifies a project was actually completed.

What the next iteration needs

In priority order: authentication, which is the precondition for deploying at all; richer assessment formats — the current multiple-choice checkpoints cannot evaluate a submitted project, which is where the real evidence of skill lives; and catalog expansion via provider APIs, at which point the retrieval stage would likely need a vector index alongside BM25 — the lexical-only tradeoff above is defensible at 61 documents and would not be at 6,000.